

План курса

План лекций

1. Объектная модель Python II.

1. Где реально хранятся атрибуты: `obj.dict`, `type(obj).dict`
2. Механизм поиска атрибутов: порядок и приоритеты (экземпляр → класс → базовые классы)
3. `getattr`, `setattr`, `hasattr` как инструменты работы с объектной моделью
4. Отличия `__getattribute__` и `__getattr__` (на уровне применения)
5. `__slots__`: зачем нужен, ограничения и эффект на память/производительность
6. Типовые ошибки: неожиданные атрибуты, опечатки, «тихие» баги без `__slots__`

2. Наследование и композиция

1. Отношения is-a и has-a;
2. Переопределение методов;
3. `super()` и MRO;
4. Множественное наследование;
5. mixins как шаблон повторного использования поведения;
6. Требования к mixin-классам;
7. Diamond проблема.

3. Duck Typing, протоколы и проверка контрактов

1. философия duck typing;
2. Неявные протоколы Python;
3. `typing.Protocol`
4. проверка контрактов: `isinstance`, `issubclass`; `@runtime_checkable`;
структурная проверка через `hasattr` и `callable`;
5. ABC vs Protocol;
6. статическая и runtime-проверка.

4. Дескрипторы и `@property`

1. протокол дескриптора (`__get__`, `__set__`, `__delete__`)
2. data и non-data descriptors;
3. функции и методы как дескрипторы;
4. `property` как частный случай;
5. инкапсуляция и защита инвариантов.

5. Итераторы и пользовательские коллекции

1. Iterable и Iterator;
 2. `__iter__` , `__next__` ;
 3. StopIteration;
 4. одноразовые и повторяемые итераторы;
 5. проектирование API коллекций.
6. Контекстные менеджеры и исключения
 1. управление ресурсами;
 2. `with` , `__enter__` , `__exit__` ;
 3. модель исключений;
 4. пользовательские исключения;
 5. contextlib;
 6. исключения как часть API.
7. Генераторы и ленивые вычисления
 1. генераторные функции и `yield`;
 2. генераторные выражения;
 3. конвейеры обработки данных;
 4. `send` , `throw` , `close` ;
 5. сравнение генераторов и коллекций.
8. Асинхронная модель Python
 1. синхронная и асинхронная модели;
 2. event loop;
 3. `async` / `await` ;
 4. `coroutine` и `task` ;
 5. асинхронные контекстные менеджеры и генераторы

Лабораторные работы.

Наличие тестов и соблюдение оформления является обязательными критериями сдачи лабораторной работы. Минимальное допустимое покрытие тестами: 80%

Предметная область:

В рамках лабораторных работ рассматривается абстрактная предметная область платформы обработки задач.

Платформа предназначена для:

- приёма задач из различных источников;
- хранения и организации задач;
- последовательной и асинхронной обработки задач;
- расширения функциональности без изменения существующего кода.

Под задачей в рамках курса понимается единица работы, которая:

- имеет идентификатор;
- содержит пользовательские данные;
- может поступать из различных источников;
- передаётся между компонентами системы и подлежит обработке.

Задача – это абстрактная единица работы, описывающая действие, которое должно быть выполнено системой или обработано прикладной логикой.

По смыслу задача ближе всего к:

- задаче из систем обработки заданий (job);
- элементу очереди задач;
- единице работы в планировщиках и background-системах (task / job / work item).

В рамках предметной области допустимы, например, такие задачи:

- «обработать заказ с указанным идентификатором»;
- «отправить уведомление пользователю»;
- «пересчитать статистику»;
- «проверить состояние ресурса»;
- «обработать входящие данные из внешнего источника».

Задача в рамках лабораторных работ:

- описывает, что нужно сделать, а не как это сделать;
- содержит данные, необходимые для выполнения работы;
- может быть создана заранее или поступать из внешнего источника;
- обрабатывается системой в синхронном или асинхронном режиме.

Лабораторная работа №1. Источники задач и контракты

Цель работы: Освоить duck typing и контрактное программирование на примере источников задач.

Постановка задачи: Необходимо реализовать подсистему приёма задач в платформе обработки задач. Задачи могут поступать из различных источников, не связанных наследованием, но обязанных реализовывать единый поведенческий контракт.

Под **задачей** в рамках данной лабораторной работы следует понимать структуру данных, содержащую только минимально необходимую информацию:

- id – уникальный идентификатор задачи;
- payload – произвольные данные задачи.

Допускается любое простое представление задачи:

- dict;
- NamedTuple;
- dataclass;
- простой пользовательский класс.

Примеры источников задач:

- задачи, загружаемые из файла;
- задачи, генерируемые программно (генератор);
- задачи, получаемые из API-заглушки; (под API-заглушкой понимается упрощённый программный компонент, имитирующий внешний источник задач)

Функциональные требования:

- все источники задач должны реализовывать единый контракт (например, метод `get_tasks()`);
- запрещено использование общего базового класса для источников;
- контракт должен быть описан с помощью `typing.Protocol`;
- должна быть реализована как минимум одна runtime-проверка соблюдения контракта;
- архитектура должна позволять добавление новых источников без изменения существующего кода;

Технические требования:

- использование `Protocol` и `runtime_checkable`;
- осмысленное применение `isinstance()` и/или `issubclass()`;
- аннотации типов для всех публичных интерфейсов;

Ожидаемый результат: Модуль приёма задач, способный работать с несколькими источниками через единый контракт.

Критерии оценки:

- корректность контракта и его применения — 40;
- архитектурная расширяемость — 20;
- качество типизации — 20;
- документация и структура кода — 20;

Лабораторная работа №2. Модель задачи: дескрипторы и `@property`

Цель работы: Освоить управление доступом к атрибутам и защиту инвариантов доменной модели.

Постановка задачи: Реализовать модель задачи (`Task`) в рамках платформы обработки задач с корректной инкапсуляцией и валидацией состояния.

Атрибуты задачи

- идентификатор задачи;
- описание задачи;
- приоритет;
- статус задачи;
- время создания;
- вычисляемые свойства (например, готовность к выполнению);

Функциональные требования:

- использование пользовательских дескрипторов для валидации атрибутов;
- использование `property` для вычисляемых или защищённых свойств;
- предотвращение некорректных состояний объекта;
- генерация специализированных исключений при нарушении инвариантов;

Технические требования:

- разделение публичного API и внутреннего состояния;
- демонстрация различий между data и non-data descriptors;
- аннотации типов и документация;

Ожидаемый результат: Класс `Task` с чистым и безопасным публичным API;

Критерии оценки:

- корректность реализации дескрипторов — 40;
- чистота и стабильность API — 30;
- обработка исключений — 20;
- документация и структура кода — 10;

Лабораторная работа №3. Очередь задач: итераторы и генераторы

Цель работы: Научиться реализовывать пользовательские коллекции и ленивую обработку задач.

Постановка задачи: Разработать очередь задач (`TaskQueue`), поддерживающую итерацию, фильтрацию и потоковую обработку задач.

Функциональные требования:

- реализация протокола итерации;
- поддержка повторного обхода очереди;
- реализация ленивых фильтров (например, по статусу и приоритету);
- корректная работа с большими объёмами задач;

Технические требования:

- использование генераторов для ленивых операций;
- отсутствие избыточного хранения данных в памяти;
- корректная обработка `StopIteration`;

Ожидаемый результат: Коллекция задач, совместимая со стандартными конструкциями Python (`for`, `list`, `sum`).

Критерии оценки:

- корректность итераторов — 40;
- использование генераторов — 30;
- эффективность решения — 20;
- документация и структура кода — 10;

Лабораторная работа №4. Асинхронный исполнитель задач

Цель работы: Научиться реализовывать приложения с асинхронной моделью управления

Постановка задачи: Реализовать асинхронный исполнитель задач, обрабатывающий задачи из очереди с использованием расширяемых обработчиков.

Функциональные требования:

- асинхронная очередь задач;
- описание контракта обработчика через `Protocol`;
- использование контекстных менеджеров для управления ресурсами;
- централизованное логирование и обработка ошибок;

Технические требования:

- корректное использование `async` / `await`;
- отсутствие блокирующих операций в event loop;
- архитектура, допускающая добавление новых типов задач и обработчиков;
- аннотации типов и документация;

Ожидаемый результат: Рабочая асинхронная система обработки задач, демонстрирующая архитектурную целостность и корректное использование механизмов Python.

Критерии оценки:

- архитектура и расширяемость — 30;
- корректность асинхронного кода — 30;
- использование контрактов — 20;
- документация и структура кода — 10;